SkillNavora

MODULE 7 HANDBOOK

Module 7: Full-Stack Application Development

5 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Module 7: Full-Stack Application Development

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 7.1 Frontend Development
2. 7.2 Backend Development
3. 7.3 Full-Stack Architecture
4. 7.4 SaaS Development
5. 7.5 Modular Application Development

MODULE 7 · SECTION 7.1

7.1 Frontend Development

DEFINITION

> The frontend is the part of the app users see and touch. HTML gives structure, CSS gives style, JavaScript gives behaviour; React builds interfaces from reusable components.

HOW IT WORKS

- Components are functions that return UI and hold state. State changes re-render only what changed.
- Forms collect input and validate it before submission; responsive design uses flexible layout so the page works on phones.
- The frontend calls the backend through HTTP and shows loading, empty and error states.

EVERY TOPIC EXPLAINED

HTML, CSS, JavaScript, TypeScript, React. HTML defines structure (headings, forms, buttons). CSS defines layout and style (flexbox, grid, colours). JavaScript/TypeScript adds behaviour. React builds the UI from components: functions that take props and return markup (JSX).

Components, state management, forms, responsive design, API integration. State (useState) holds data that changes; when state changes React re-renders. State management for larger apps uses context, Zustand or Redux. Forms collect and validate input. Responsive design adapts layout to screen size with CSS media queries or Tailwind. API integration calls the backend with fetch, showing loading, success and error states.

WORKED EXAMPLE

> A lead form validates the email in the browser, shows a spinner while submitting, and displays the server's error message if it fails.

RESOURCES

- React: learn (official) https://react.dev/learn
- MDN: HTML and CSS https://developer.mozilla.org/en-US/docs/Learn_web_development/Core

7.2 Backend Development

DEFINITION

> The backend runs business logic, enforces rules, talks to databases and external APIs, and returns data. FastAPI is a fast Python framework for building JSON APIs.

HOW IT WORKS

- Routes map URLs and methods to Python functions. Pydantic models validate request bodies and shape responses.
- Middleware runs on every request (logging, auth). Background tasks run work after the response is sent.
- async def lets the server handle many waiting requests at once, useful for calls to LLM APIs.

EVERY TOPIC EXPLAINED

Backend fundamentals; Python application development; FastAPI. The backend receives requests, applies business logic, talks to databases and services, and returns responses. Python is a strong backend choice for AI because of its libraries. FastAPI is a modern Python framework: fast, typed, async, with automatic interactive docs at /docs.

Routes, endpoints, request and response handling, middleware, validation. A route (endpoint) is a URL + method handled by a function (@app.get('/leads/{id}')). Request handling reads path, query and body parameters; response handling returns JSON and status codes. Middleware runs on every request (CORS, logging, auth). Validation uses Pydantic models that reject bad input with clear 422 errors.

Error handling, background tasks, async programming. Error handling raises HTTPException(status_code=404, detail='Lead not found') and logs unexpected errors. Background tasks run work after responding (sending an email). Async programming (async def, await) lets one server handle many requests while waiting for databases or AI APIs.

WORKED EXAMPLE

> @app.post('/leads') validates a Lead model, saves it, queues an enrichment task, and returns 201 with the new ID.

RESOURCES

- FastAPI: first steps (official) https://fastapi.tiangolo.com/tutorial/first-steps/
- FastAPI: tutorial index https://fastapi.tiangolo.com/tutorial/

7.3 Full-Stack Architecture

DEFINITION

> Full-stack architecture links the frontend, the backend, the database and the services they use, with authentication and clear API contracts between them.

HOW IT WORKS

- Authentication proves identity (login); authorisation decides what that identity may do (roles).
- Sessions or tokens (JWT, OAuth) carry identity between requests. Store passwords hashed, never in plain text.
- File uploads go to object storage with size and type checks; logs record requests and errors without secrets.

EVERY TOPIC EXPLAINED

Client-server architecture; REST APIs. Client-server architecture: the client (browser, mobile app) sends requests; the server processes them and responds. REST APIs define the contract between them with resources, methods and status codes.

Authentication, authorisation, session management, database connectivity. Authentication answers 'who are you?' (passwords, magic links, Google login, tokens). Authorisation answers 'what may you do?' (roles, permissions, row-level security). Session management keeps users logged in with secure cookies or short-lived tokens (JWT) plus refresh tokens. Database connectivity uses connection pools and an ORM (SQLAlchemy) or a query builder.

File uploads, third-party integrations, application logging. File uploads: check type and size, store in object storage (S3, Supabase Storage), save only the reference in the database. Third-party integrations: payments, email, CRM, AI APIs, each wrapped in a small service module. Application logging: structured logs (JSON) with request IDs, without passwords or personal data.

WORKED EXAMPLE

> A user logs in, receives a short-lived token, and the API checks the token and the user's role before returning another company's data is refused.

RESOURCES

- Supabase Auth docs https://supabase.com/docs/guides/auth
- OWASP Authentication Cheat Sheet https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

7.4 SaaS Development

DEFINITION

> SaaS is software sold as a subscription and used online. A multi-tenant app serves many customer organisations from one codebase while keeping their data separate.

HOW IT WORKS

- Tenancy: each row carries an organisation ID, and every query filters on it; row-level security enforces this in the database.
- Roles (admin, member, viewer) control access. Subscriptions and billing use a payment provider's webhooks to switch plans on or off.
- Usage tracking counts API calls or seats to enforce plan limits; onboarding and product analytics show where customers get stuck.

EVERY TOPIC EXPLAINED

What SaaS is; multi-tenant architecture. SaaS (software as a service) is software delivered over the internet on a subscription. Multi-tenant architecture serves many customers (tenants) from one deployment, keeping each tenant's data isolated, usually with a tenant_id column and row-level security, or a separate schema or database per tenant.

User and organisation management; role-based access control. User management: sign-up, invite, profile, password reset. Organisation management: workspaces with members. Role-based access control (RBAC): roles such as owner, admin, member, viewer map to permissions checked on every request.

Subscriptions, payments, usage tracking, API limits, billing. Subscriptions and payments use a provider such as Stripe or Razorpay: plans, checkout, invoices and webhooks that update the account's plan. Usage tracking counts metered units (AI calls, documents). API limits enforce plan quotas. Billing combines subscription fees and usage.

Admin dashboards, customer onboarding, product analytics. Admin dashboards let your team manage accounts, plans and support. Customer onboarding guides new users to first value (checklists, sample data). Product analytics track sign-ups, activation, feature usage and retention.

WORKED EXAMPLE

> A team plan allows 5 seats and 10,000 AI calls a month; the app counts calls per organisation and blocks new calls with an upgrade prompt at the limit.

RESOURCES

- Stripe billing docs https://docs.stripe.com/billing
- Supabase: multi-tenancy with RLS https://supabase.com/docs/guides/database/postgres/row-level-security

7.5 Modular Application Development

DEFINITION

> A modular application splits the system into parts with clear boundaries (modules or services) so each can change without breaking the others.

HOW IT WORKS

- Layers: API layer (routes), service layer (business rules), data layer (database access). Each depends only on the layer below.
- Configuration comes from environment variables, not code, so the same build runs in test and production.
- Scale the parts that need it; keep interfaces stable so teams can work in parallel.

EVERY TOPIC EXPLAINED

Modular architecture; reusable components; service separation. Modular architecture splits the app into modules with clear responsibilities (auth, billing, documents, AI). Reusable components (UI and backend functions) are written once and used in many places. Service separation keeps different concerns in different modules or services.

API and database layers; configuration; environment variables. The API layer handles HTTP; the service layer holds business logic; the database layer handles queries. Configuration (URLs, limits, feature flags) lives in config files or environment variables. Environment variables hold settings and secrets that differ between development and production; never commit them to Git (.env in .gitignore).

Scalability and maintainability. Scalability: the app handles more users by adding instances (stateless servers), caching and queues. Maintainability: clean modules, tests and documentation let new developers change code safely.

WORKED EXAMPLE

> The billing module exposes one function, charge_customer(), used by both the web API and the nightly job; the payment provider can be swapped inside that module.

RESOURCES

- FastAPI: bigger applications https://fastapi.tiangolo.com/tutorial/bigger-applications/
- The Twelve-Factor App (config) https://12factor.net/config